NoteVision AI: Handwritten Text Recognition Using Deep Learning

A Project Report Submitted in
the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

Sneha Bomma	**:**	**2420030231**
Kola Hitheshini	**:**	**2420030233**
Lalitha mahisri M	**:**	**2420030445**

Under the Esteemed Guidance of

Dr. N. Sunanda

Associate Professor

Department of Computer Science and Engineering

AY- 2026-2027

Semester: 3-1

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled "**NoteVision AI: Handwritten Text Recognition Using Deep Learning**" is a record of Bonafide work of **Sneha Bomma – 2420030231, Kola Hitheshini – 2420030233, and Lalitha Mahisri M – 2420030445**, submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to K L (Deemed to be) University. The work presented in this report is based on the project developed by the team.

Sneha Bomma –2420030231

Kola Hitheshini - 2420030233

Lalitha mahisri M - 2420030445

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the mini project based report entitled "**NoteVision AI: Handwritten Text Recognition Using Deep Learning**" is a record of Bonafide work of **Sneha Bomma – 2420030231, Kola Hitheshini – 2420030233, and Lalitha Mahisri M – 2420030445**, in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of COMPUTER SCIENCE AND ENGINEERING, K L (Deemed to be) University, during the academic year **2026–2027**.

Signature of the Guide	Course Coordinator	Head of the Department
Dr. N. Sunanda	Dr. N. Sunanda	Dr.P. Venkateswara Rao
Associate Professor, CSE	Department of CSE	Department of CSE

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator Dr N. Sunanda, and my guide **Dr N. Sunanda,** Department of Computer Science and Engineering which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr Venkateswara Rao Pulipati,** Head of the ***Department for COMPUTER SCIENCE AND ENGINEERING*** for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

Sneha Bomma –2420030231

Kola Hitheshini - 2420030233

Lalitha mahisri M - 2420030445

Abstract

Handwritten notes remain widely used in education and everyday work, but paper-based information is difficult to search, edit, store, and share. NoteVision AI is a hybrid handwritten-note digitization system that converts uploaded handwritten note images into editable digital text and allows users to export the corrected output as a PDF. The system combines an EMNIST-trained Convolutional Neural Network (CNN) for character-level classification with Microsoft TrOCR for line-level handwriting recognition. OpenCV is used for image preprocessing and segmentation, TensorFlow loads the trained EMNIST model, Hugging Face Transformers provides the TrOCR model, and Streamlit presents the application interface.

The workflow includes image upload, grayscale conversion, noise reduction, contrast enhancement, thresholding, character and line segmentation, EMNIST-based character prediction, TrOCR-based line recognition, text reconstruction, confidence review, and PDF generation. EMNIST provides character-level visual classification using 28 × 28 handwritten character images, while TrOCR processes complete text-line images and uses contextual information to produce more readable words and sentences. The recognized text is displayed in an editable interface so that users can correct errors before downloading the final PDF.

The hybrid approach addresses the limitations of isolated-character recognition when processing natural handwritten notes containing connected letters, variable spacing, and multiple lines. The system retains EMNIST as an explicit character-classification and confidence-analysis component while using TrOCR as a complementary sequence-level recognizer. NoteVision AI demonstrates an integrated workflow for transforming handwritten notes into editable and shareable digital documents.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	5
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objectives 1.5 Importance of the application 1.6 Target users/audience	8-9
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	10
3	Technology Stack	3.1 Python 3.2 Streamlit 3.3 OpenCV 3.4 TensorFlow and EMNIST CNN 3.5 Microsoft TrOCR 3.6 Dataset and external models	11-12
4	System Architecture	4.1 High-level architecture 4.2 Description of each layer 4.3 Deployment architecture	13-14
5	Design	5.1 Workflow design 5.2 Data and model design 5.3 Interface design	15
6	Implementation	6.1 Module-wise implementation 6.2 Image preprocessing and segmentation 6.3 EMNIST CNN classification 6.4 TrOCR recognition 6.5 Text reconstruction and PDF generation 6.6 Authentication and authorization	16-17
7	Features	7.1 Main features 7.2 How the features work	18
8	Testing	8.1 Testing approach 8.2 Functional test cases 8.3 Integration testing 8.4 Tools used for testing 8.5 Results	19-20
9	Deployment	9.1 Application execution 9.2 Environment configuration 9.3 Hosting	21
10	Challenges & Limitations	10.1 Issues faced 10.2 Solutions applied 10.3 Current limitations	22
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations and optimizations	23
12	Conclusion	12.1 Summary 12.2 Achievements 12.3 Skills learned	24
13	References	Research papers, dataset documentation, model card,	25

		and technical Documentation	
14	Appendices	14.1 Application modules 14.2 Model and dataset details 14.3 Installation and execution requirements 14.4 Project team 14.5 Supporting material	26

CHAPTER 1

INTRODUCTION

1.1 Background of the Project

Handwritten notes are an important part of academic learning, personal organization, and professional work. Students and professionals often record information on paper because handwriting is convenient and flexible. However, handwritten documents are difficult to search, edit, organize, and share. Converting these notes into machine-readable text makes the information easier to preserve and reuse.

Optical Character Recognition (OCR) is the process of converting text present in images into digital text. Printed text recognition is comparatively easier because printed characters usually have consistent shapes and spacing. Handwriting recognition is more challenging because writing styles vary from person to person. Differences in stroke thickness, character size, spacing, alignment, lighting, noise, and connected letters can affect recognition.

NoteVision AI is designed as a hybrid handwritten-note digitization application. It combines an EMNIST-trained Convolutional Neural Network for character-level classification with Microsoft TrOCR for complete text-line recognition. Image processing and segmentation are performed using OpenCV, while Streamlit provides the user interface.

1.2 Problem Statement

Paper-based handwritten notes are not directly searchable or editable. Manually typing handwritten content into a computer is time-consuming and may introduce transcription errors. Conventional character-level recognition also faces difficulty when letters are connected or when complete words and sentences must be reconstructed from isolated character predictions.

The project addresses the problem of converting uploaded handwritten note images into editable digital text. It aims to combine character-level classification, line-level recognition, confidence review, and PDF generation in a single workflow.

1.3 Scope of the Project

The scope of NoteVision AI includes accepting a handwritten note image, preparing the image for recognition, identifying candidate character and line regions, classifying character samples using an EMNIST-trained CNN, recognizing complete text lines using Microsoft TrOCR, reconstructing the text in reading order, displaying the result for user correction, and generating a downloadable PDF.

The prototype focuses on image-based handwritten notes. Recognition quality depends on image clarity, handwriting style, preprocessing, and segmentation. The system is not claimed to provide perfect recognition for every handwriting style or document layout.

1.4 Objectives

1. To develop a user-friendly application for digitizing handwritten notes.
2. To preprocess uploaded images to improve the contrast between handwriting and background.
3. To use an EMNIST-trained CNN for character-level handwritten classification.
4. To use Microsoft TrOCR for line-level handwriting recognition.
5. To reconstruct recognized text in the correct reading order.
6. To provide confidence information and an editable text area for manual correction.
7. To generate a downloadable PDF containing the reviewed digital text.

1.5 Importance of the Application

Digitizing handwritten notes helps users preserve information, edit content, search through text, and share documents conveniently. NoteVision AI reduces the need to manually retype every handwritten page and provides a workflow in which machine recognition is followed by human review. The hybrid design also demonstrates how a traditional character-classification model can be combined with a Transformer-based OCR model.

1.6 Target Users / Audience

The intended users include students who want to digitize lecture notes, teachers and researchers who maintain handwritten material, and individuals who want to convert personal handwritten notes into editable documents. The prototype can also serve as an academic demonstration of computer vision, deep learning, OCR, and application integration.

CHAPTER 2

SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The application requires a computer capable of running Python and the required machine-learning libraries. A standard laptop or desktop with sufficient memory and storage can be used for development and testing. An image input device such as a mobile phone camera or scanner may be used to capture handwritten notes. A GPU may improve model inference speed where supported, but the project description does not specify a mandatory GPU configuration.

2.2 Software Requirements

- Operating system: Windows, Linux, or another Python-compatible environment.
- Programming language: Python.
- User interface: Streamlit.
- Image processing: OpenCV.
- Character-classification framework: TensorFlow.
- OCR framework: Hugging Face Transformers.
- Handwriting recognition model: microsoft/trocr-base-handwritten.
- Dataset: EMNIST for handwritten character classification.
- PDF generation: a PDF generation module integrated into the application.

2.3 Development Tools and Frameworks

Python is used to coordinate the application workflow and model inference. Streamlit provides the interface for image upload, displaying processed images, presenting recognized text, and offering the PDF download option. OpenCV performs image preprocessing and segmentation. TensorFlow loads the trained EMNIST CNN. Hugging Face Transformers loads Microsoft TrOCR and performs line-level recognition. The modular implementation allows preprocessing, segmentation, recognition, and document generation to be improved independently.

CHAPTER 3

TECHNOLOGY STACK

3.1 Programming Language – Python

Python is used to implement the application pipeline and connect the image-processing, machine-learning, OCR, interface, and PDF-generation components. Its library ecosystem supports rapid development of computer-vision and deep-learning applications.

3.2 Front-end – Streamlit

Streamlit is used to create the application interface. It manages the upload of handwritten note images and displays the original image, processed image, detected regions, recognized text, confidence information, and PDF download option. It provides an interactive workflow without requiring a separate front-end framework.

3.3 Image Processing – OpenCV

OpenCV is used to prepare handwritten images for recognition. The operations described in the project include grayscale conversion, noise reduction, contrast enhancement, thresholding, inversion when required, and extraction of candidate character and line regions. Bounding boxes are used to record the location of detected regions.

3.4 Machine Learning – TensorFlow and EMNIST CNN

The EMNIST dataset contains handwritten character images represented as 28 × 28 grayscale samples. The CNN is trained to learn visual patterns such as strokes, edges, shapes, and character structures. During inference, normalized character images are passed to the model, which returns class probabilities. The class with the highest probability is treated as the predicted character, while alternative probabilities support confidence review.

3.5 OCR – Microsoft TrOCR

The project uses the microsoft/trocr-base-handwritten checkpoint through the Hugging Face Transformers library. TrOCR uses a Transformer-based image encoder and text decoder to generate a text sequence from a text-line image. The model is designed for single-line handwritten text recognition and complements the EMNIST character classifier by using word- and line-level context.

3.6 Dataset and External Models

EMNIST is used as the foundation for training the character classifier. It extends MNIST with handwritten letters and digits and provides multiple dataset splits. The selected split determines the character classes available to the classifier. The TrOCR checkpoint is a separately pretrained handwriting recognition model fine-tuned on IAM handwriting data, as described in the project paper.

3.7 Version Control and Supporting Resources

The project paper identifies the use of Python libraries and pretrained model resources. A specific version-control workflow, repository branching strategy, or deployment service is not detailed in the supplied research paper and is therefore not asserted in this report.

CHAPTER 4

SYSTEM ARCHITECTURE

4.1 High-Level Architecture

The NoteVision AI architecture consists of seven major stages: image upload, preprocessing, character and line segmentation, EMNIST CNN classification, TrOCR line recognition, text reconstruction and confidence review, and PDF generation. The Streamlit application coordinates these stages and presents the output to the user.

1. **User uploads handwritten image**
2. **Streamlit application**
3. **OpenCV preprocessing**
4. **Character and line segmentation**
5. **EMNIST CNN classification and TrOCR line recognition**
6. **Text reconstruction and confidence review**
7. **Editable text**
8. **PDF generation.**

4.2 Description of Each Layer

Input layer: Accepts an uploaded handwritten note image through the Streamlit interface.

Preprocessing layer: Converts the image into a suitable representation by applying grayscale conversion, noise reduction, contrast enhancement, thresholding, and inversion when necessary.

Segmentation layer: Uses OpenCV-based image analysis to detect candidate character regions and text-line regions. Bounding boxes are grouped and ordered to preserve approximate reading order.

Recognition layer: The EMNIST CNN classifies normalized character crops and provides class probabilities. Microsoft TrOCR processes segmented line images and generates complete text sequences.

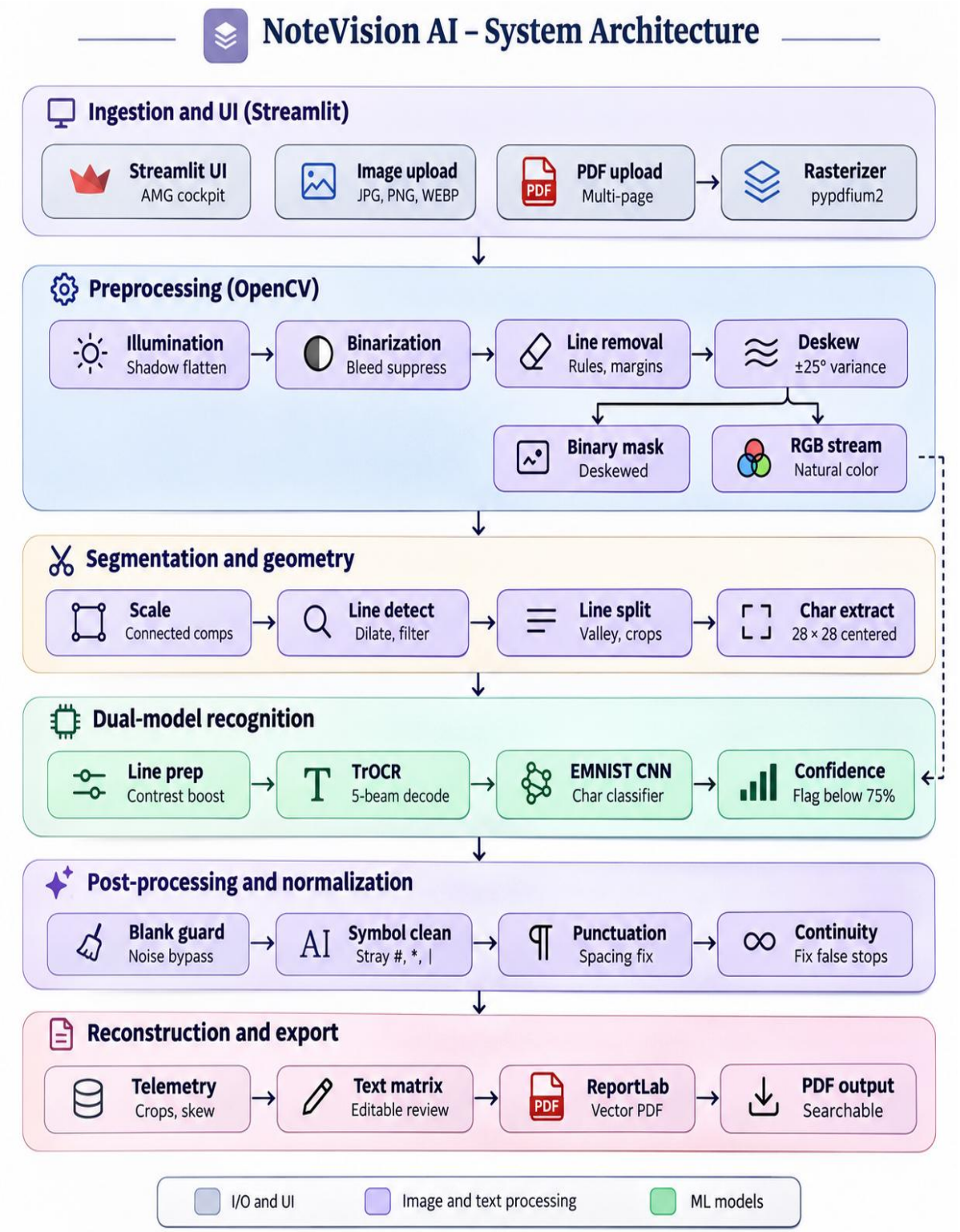
Reconstruction and review layer: Recognized lines are arranged from top to bottom. The application displays the text in an editable area and presents confidence information so users can review and correct uncertain output.

Output layer: The corrected text is passed to the PDF generation module to create a downloadable digital document.

4.3 Deployment Architecture

The supplied project paper describes a Streamlit-based prototype and its model-loading workflow. It does not specify a production hosting provider, cloud infrastructure, CI/CD pipeline, or public deployment URL. Accordingly, this report describes the application

architecture without claiming a particular cloud deployment.



CHAPTER 5

DESIGN

5.1 Workflow Design

The application follows a sequential image-to-text workflow. First, the user uploads a handwritten note image. The application preprocesses the image and extracts candidate character and line regions. The EMNIST CNN performs character-level classification, while TrOCR recognizes complete text lines. The output is reconstructed in reading order and shown to the user for correction. Finally, the reviewed text is exported as a PDF.

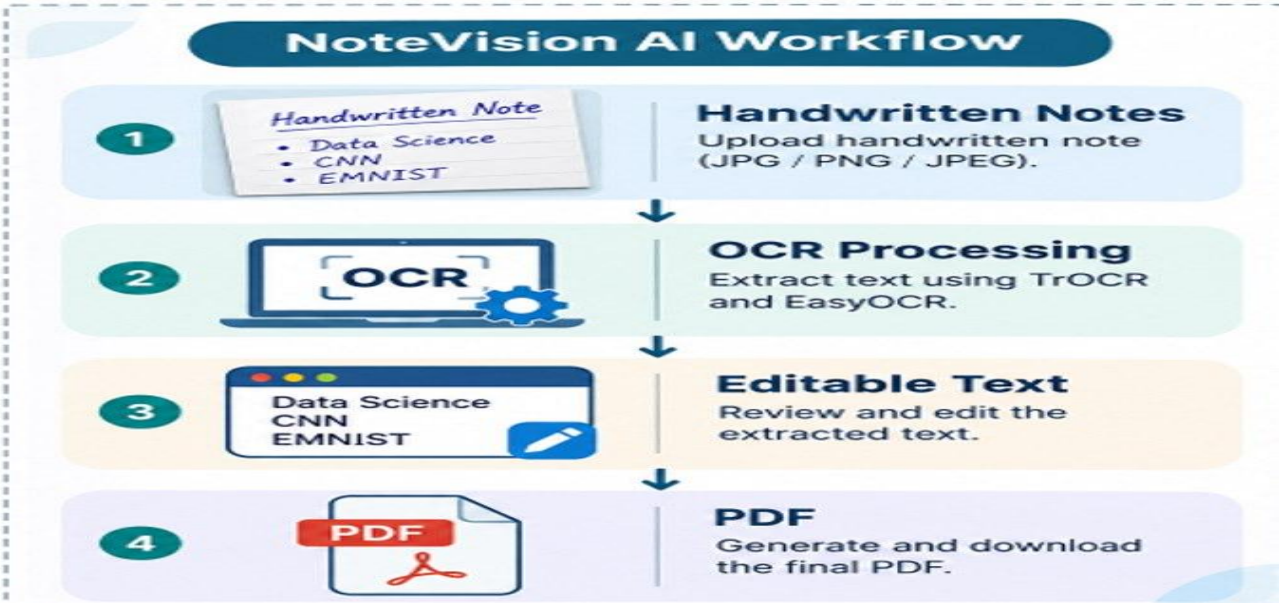
5.2 Data and Model Design

The EMNIST dataset supplies 28 × 28 grayscale handwritten character samples for CNN training. Pixel values are normalized and labels are prepared for multiclass classification. Orientation correction is important because EMNIST samples may appear rotated and transposed in their stored representation.

The TrOCR component uses the microsoft/trocr-base-handwritten model. It accepts segmented line images and generates text sequences. The two models have separate responsibilities: EMNIST contributes character-level visual classification and confidence analysis, while TrOCR contributes context-aware line-level recognition.

5.3 Interface Design

The Streamlit interface manages the user interaction and displays the original uploaded image, processed image, detected regions, recognized text, confidence information, and PDF download option. The editable text area provides a human-in-the-loop correction stage before document generation.



CHAPTER 6

IMPLEMENTATION

6.1 Module-Wise Implementation

1. Image upload module: receives the handwritten note image from the user.
2. Preprocessing module: reads the image and applies grayscale conversion, noise reduction, contrast enhancement, thresholding, and optional inversion.
3. Segmentation module: detects candidate character regions, records bounding boxes, and groups regions into text lines.
4. EMNIST recognition module: loads the trained TensorFlow CNN, normalizes character images, predicts character classes, and retains class probabilities.
5. TrOCR recognition module: loads microsoft/trocr-base-handwritten through Transformers and recognizes complete segmented text lines.
6. Reconstruction module: combines recognized lines in top-to-bottom order and prepares the output for review.
7. PDF module: converts the corrected text into a downloadable PDF.

6.2 Image Preprocessing and Segmentation

The uploaded image is converted into a form suitable for analysis. Preprocessing aims to maximize contrast between handwritten strokes and the background. OpenCV-based analysis identifies candidate handwritten regions and creates bounding boxes. Each character candidate is cropped, normalized, and resized to 28 × 28 pixels for compatibility with the EMNIST CNN. Line grouping is necessary because sorting all boxes only from left to right could mix characters from different lines.

6.3 EMNIST CNN Classification

Each normalized character image is passed to the trained CNN. The model produces probabilities for the available character classes. The highest-probability class is selected as the predicted character, and alternative class probabilities are retained for confidence analysis. The model learns spatial features through convolutional layers and performs multiclass classification.

6.4 TrOCR-Based Line Recognition

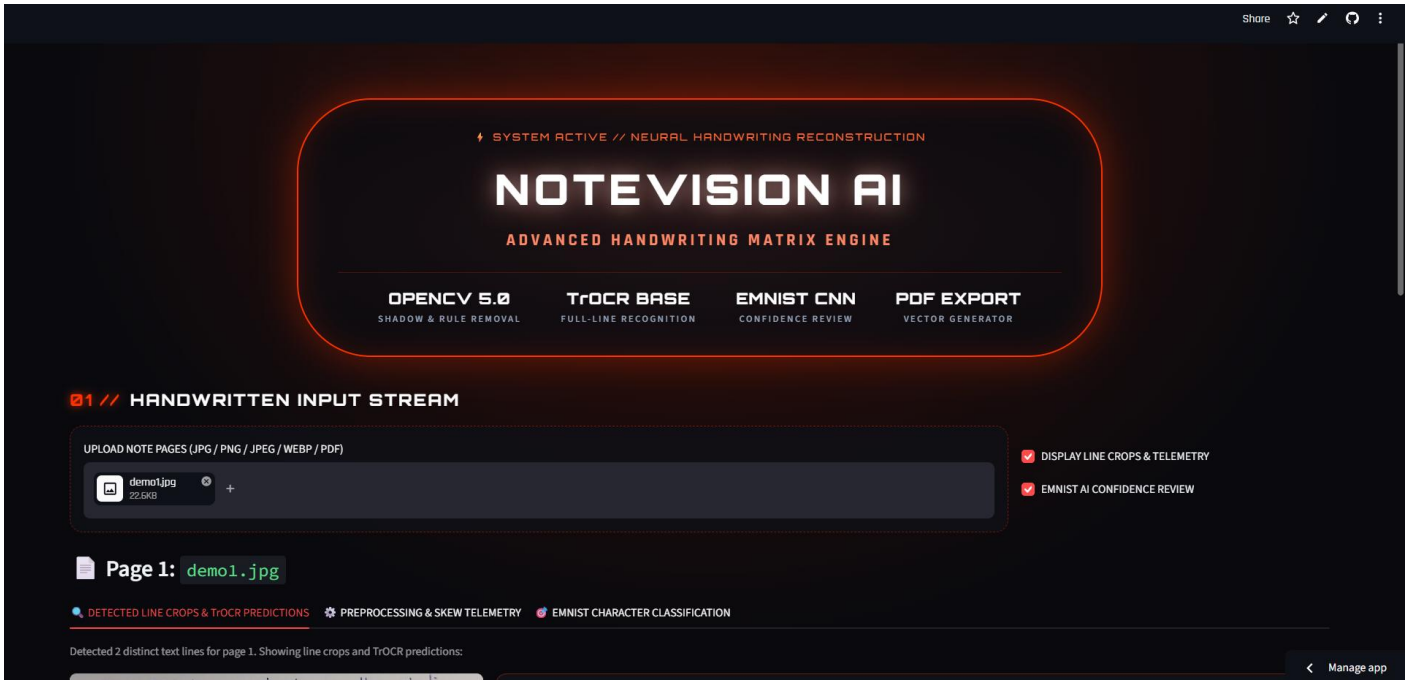
Segmented line images are passed to Microsoft TrOCR. The Transformer-based vision encoder and text decoder map each line image to a generated text sequence. Unlike isolated-character prediction, line-level recognition can use contextual information when generating words. This helps address errors caused by visually similar characters and word formation.

6.5 Text Reconstruction and PDF Generation

The recognized line outputs are combined in top-to-bottom order. The application displays the reconstructed text in an editable text area. Users can correct recognition errors before exporting. The PDF generation module creates a digital document containing the reviewed text rather than forcing users to accept uncorrected model output.

6.6 Authentication and Authorization

The supplied project paper does not describe a login system, user roles, authentication mechanism, or authorization rules. These features are not represented as implemented functionality in this report.



CHAPTER 7

FEATURES

7.1 Main Features

• Upload handwritten note images.
• Preprocess images to improve readability for recognition.
• Detect candidate handwritten character and line regions.
• Classify isolated character regions using the EMNIST-trained CNN.
• Recognize complete text lines using Microsoft TrOCR.
• Reconstruct recognized text in reading order.
• Display confidence information and alternative character predictions.
• Allow users to edit and correct recognized text.
• Generate and download a PDF containing the reviewed text.

7.2 How the Features Work

Image upload and preprocessing: The user selects a handwritten note image. The application prepares the image by reducing noise and improving contrast.

Character and line detection: OpenCV identifies candidate text regions and records their bounding boxes. Regions are grouped into lines to maintain reading order.

Hybrid recognition: The EMNIST CNN classifies normalized character images and provides class probabilities. TrOCR recognizes segmented text lines as sequences, using context to improve word-level readability.

Confidence review and editing: The application presents the reconstructed text and confidence information. Low-confidence predictions can be reviewed and corrected by the user.

PDF export: After editing, the user generates a PDF containing the corrected digital text.

CHAPTER 8

TESTING

8.1 Testing Approach

Testing of the prototype should examine the complete image-to-text workflow, including image upload, preprocessing, segmentation, model loading, recognition, text reconstruction, editing, and PDF generation. The supplied paper discusses practical recognition challenges but does not provide a formal test-case table with measured pass/fail results. The cases below describe functional checks appropriate to the documented workflow; they should not be interpreted as reported experimental results.

8.2 Functional Test Cases

Test Case 1 – Image upload: Provide a supported handwritten image and verify that it is displayed by the application.

Test Case 2 – Preprocessing: Verify that grayscale conversion, noise reduction, contrast enhancement, and thresholding are applied to the uploaded image.

Test Case 3 – Segmentation: Verify that candidate character and line regions are detected and ordered by their page positions.

Test Case 4 – EMNIST inference: Verify that normalized character crops are passed to the CNN and class probabilities are produced.

Test Case 5 – TrOCR inference: Verify that segmented line images are passed to the TrOCR model and text sequences are returned.

Test Case 6 – Text reconstruction: Verify that recognized lines are combined in top-to-bottom reading order.

Test Case 7 – Text editing: Verify that the displayed output can be corrected by the user.

Test Case 8 – PDF generation: Verify that the reviewed text can be exported as a downloadable PDF.

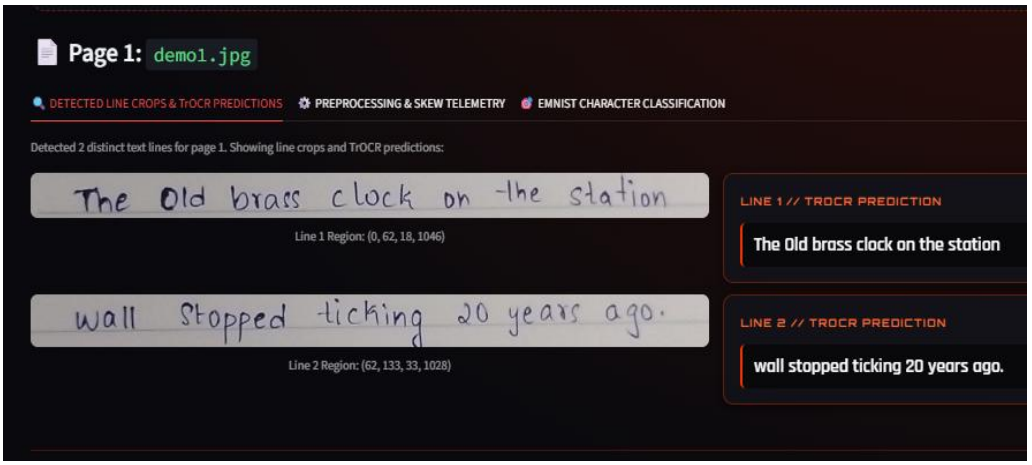
8.3 Integration Testing

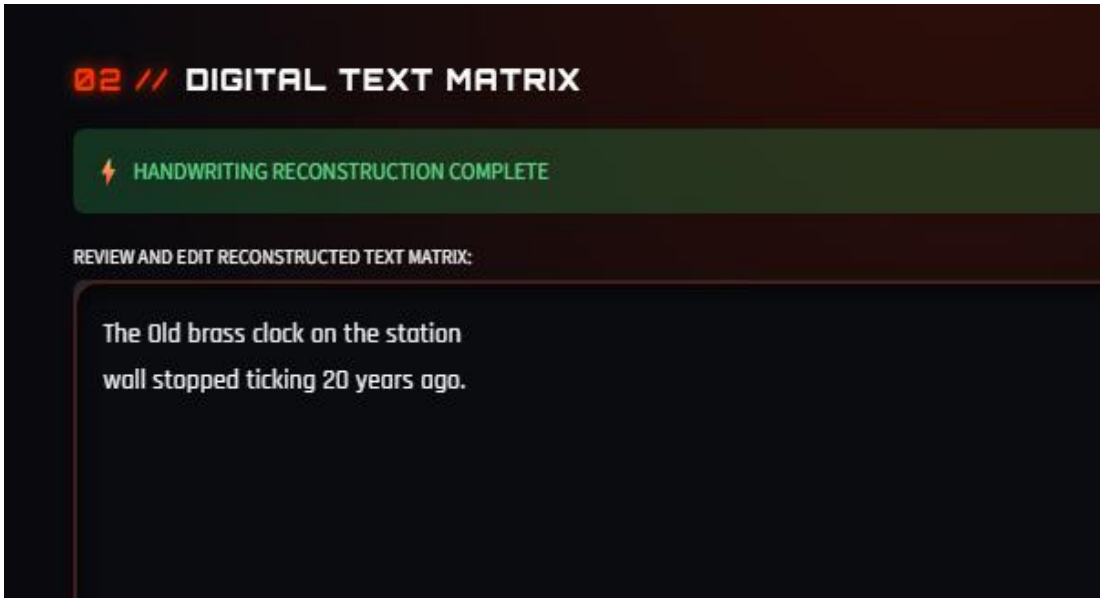
Integration testing focuses on the interaction between OpenCV preprocessing and segmentation, TensorFlow-based EMNIST inference, Hugging Face Transformers-based TrOCR inference, the Streamlit interface, and the PDF generation module. The modular structure allows individual stages to be tested and improved without redesigning the complete application.

8.4 Tools Used for Testing

The project uses the application interface to exercise the end-to-end workflow. OpenCV, TensorFlow, and Transformers are the primary software components involved in processing and recognition. The supplied research paper does not identify a separate testing framework or provide Postman-based API tests; no such testing is claimed here.

8.5 Results


Page 1: demo1.jpg
DETECTED LINE CROPS & TrOCR PREDICTIONS
PREPROCESSING & SKEW TELEMETRY
EMNIST CHARACTER CLASSIFICATION
Detected 2 distinct text lines for page 1. Showing line crops and TrOCR predictions:
The Old brass clock on the station
Line 1 Region: (0, 62, 18, 1046)
LINE 1 // TROCR PREDICTION
The Old brass clock on the station
wall stopped ticking 20 years ago.
Line 2 Region: (62, 133, 33, 1028)
LINE 2 // TROCR PREDICTION
wall stopped ticking 20 years ago.


02 // DIGITAL TEXT MATRIX
HANDWRITING RECONSTRUCTION COMPLETE
REVIEW AND EDIT RECONSTRUCTED TEXT MATRIX:
The Old brass clock on the station
wall stopped ticking 20 years ago.

CHAPTER 9

DEPLOYMENT

9.1 Application Execution

The project paper describes a Streamlit application that coordinates image upload, preprocessing, segmentation, recognition, text review, and PDF generation. The application requires its Python dependencies, trained EMNIST model, and TrOCR checkpoint to be available in the execution environment.

9.2 Environment Configuration

The environment must include Python and the required libraries: Streamlit, OpenCV, TensorFlow, Hugging Face Transformers, and the dependencies needed by the selected TrOCR checkpoint and PDF generation module. The trained EMNIST model must be available at the configured model path. The TrOCR checkpoint is loaded using the Transformers library.

9.3 Hosting of Front-end and Back-end

Our project uses Streamlit to provide a simple and interactive user interface. Currently, the application is designed to run locally using Python and the required libraries. We have not configured cloud hosting or CI/CD pipelines yet, as our main focus was on developing and testing the core functionality of NoteVision AI.

CHAPTER 10

CHALLENGES AND LIMITATIONS

10.1 Issues Faced During Development

A major challenge is the domain gap between EMNIST and real handwritten notes. EMNIST primarily contains isolated character samples, whereas natural notes contain connected letters, variable spacing, complete words, multiple lines, diagrams, and uneven illumination.

Segmentation is another challenge. Incorrect bounding boxes can split one character into multiple regions or combine multiple characters into one region. Incorrect line grouping can also disturb reading order. Visually similar characters, such as O and 0 or I and l, may be confused depending on the handwriting style.

10.2 Solutions Applied

The proposed workflow addresses these issues through image preprocessing, line-aware bounding boxes, EMNIST classification, TrOCR line recognition, confidence analysis, and user editing. TrOCR provides sequence-level recognition for complete text lines, while EMNIST remains part of the system for character-level visual classification and confidence information. The editable output allows users to correct errors before PDF generation.

10.3 Current Limitations

Recognition quality depends on the input image, handwriting style, preprocessing quality, and segmentation accuracy. TrOCR also depends on the quality of line segmentation and may not recognize every writing style correctly. The system may confuse visually similar characters and may require manual correction. The supplied paper does not report a quantified accuracy score or a comprehensive benchmark across handwriting types.

CHAPTER 11

FUTURE ENHANCEMENTS

11.1 Planned Features

Future work can fine-tune TrOCR on a larger collection of real academic handwritten notes and train the EMNIST classifier with additional handwriting samples. Data augmentation can be applied to improve robustness. Line and word segmentation can be enhanced to reduce errors caused by connected characters and inconsistent spacing.

11.2 Possible Integrations and Optimizations

Language-based post-processing can be used to correct improbable character sequences after recognition. A domain-specific language model may be integrated after TrOCR while retaining EMNIST as the core visual classification component. Further enhancements include multi-page document support, automatic paragraph detection, layout preservation, and joint confidence estimation between EMNIST and TrOCR.

CHAPTER 12

CONCLUSION

12.1 Summary of the Project

NoteVision AI presents a hybrid handwritten-note digitization system that combines EMNIST-based CNN image classification with Microsoft TrOCR line-level handwriting recognition. The application includes image preprocessing, OpenCV-based segmentation, character classification, Transformer-based sequence recognition, ordered text reconstruction, confidence review, and PDF generation.

12.2 Achievements

The project brings character-level and line-level recognition into one workflow. EMNIST contributes visual classification and confidence analysis for segmented characters, while TrOCR recognizes complete handwritten lines using contextual information. The Streamlit interface allows users to inspect and edit the recognized text before generating a downloadable PDF.

12.3 Skills Learned During Development

The project provides practical experience in Python application development, image preprocessing, OpenCV-based segmentation, CNN-based image classification, dataset preparation, Transformer-based OCR, integration of pretrained models, confidence analysis, user-interface development with Streamlit, and PDF generation.

CHAPTER 13

REFERENCES

[1] G. Cohen, S. Afshar, J. Tapson, and A. van Schaik, "EMNIST: an extension of MNIST to handwritten letters," 2017 International Joint Conference on Neural Networks (IJCNN), 2017. doi: 10.1109/IJCNN.2017.7966217.

[2] TensorFlow Datasets, "EMNIST Dataset.Available: https://www.tensorflow.org/datasets/catalog/emnist

[3] S. L. Chooi and A. S. B. Ab Ghafar, "Handwritten Character Recognition Using Convolutional Neural Network," Progress in Engineering Application and Technology, 2021. Available: https://periodical.uthm.edu.my/index.php/peat/article/view/973

[4] J. Jo, H. I. Koo, J. W. Soh, and N. I. Cho, "Handwritten Text Segmentation via End-to-End Learning of Convolutional Neural Network," arXiv:1906.05229, 2019. Available: https://arxiv.org/abs/1906.05229

[5] TensorFlow, "EMNIST character recognition documentation.Available: https://www.tensorflow.org/federated/api_docs/python/tff/simulation/baselines/emnist/create_character_recognition_task

[6] [1]Hugging Face, "microsoft/trocr-base-handwritten," Model Card. Available: https://huggingface.co/microsoft/trocr-base-handwritten

[7] M. Li et al., "TrOCR: Transformer-based Optical Character Recognition with Pre-trained Models," arXiv:2109.10282, 2021. Available: https://arxiv.org/abs/2109.10282

CHAPTER 14

APPENDICES

14.1 Application Modules

The application is organized into modular components: image preprocessing, character and line segmentation, EMNIST classification, TrOCR line recognition, text reconstruction and confidence review, Streamlit interface management, and PDF generation. Each module performs a separate task and can be enhanced independently.

14.2 Model and Dataset Details

Dataset: EMNIST handwritten character images, represented as 28 × 28 grayscale samples. The selected split determines the available character classes.

Character model: A trained Convolutional Neural Network implemented using TensorFlow. It produces class probabilities for segmented character images.

Line recognition model: microsoft/trocr-base-handwritten, loaded using the Hugging Face Transformers library. The model is fine-tuned on IAM handwriting data and is intended for single-line OCR.

14.3 Installation and Execution Requirements

1. Install Python in the development environment.
2. Install the project dependencies required for Streamlit, OpenCV, TensorFlow, Transformers, and PDF generation.
3. Place the trained EMNIST CNN model in the configured model directory.
4. Ensure the TrOCR checkpoint can be loaded by the Transformers library.
5. Start the Streamlit application using the project's application entry point.
6. Upload a handwritten note image, review the recognized text, make corrections, and generate the PDF.

14.4 Project Team

Sneha Bomma – 2420030231
Kola Hitheshini – 2420030233
Lalitha Mahisri M– 2420030445

Project Guide: Dr. N. Sunanda, Associate Professor, Department of CSE.

14.5 Supporting Material

The project paper provides the system architecture, methodology, dataset description, model details, implementation overview, challenges, future enhancements, conclusion, and research references. Application screenshots, source-code listings, and measured test results should be added to this appendix when the final implementation artifacts are available. They have not been fabricated in this report.